

DOCUMENTAÇÃO DE LABORATÓRIO

Gerenciar o Armazenamento

Snapshots EBS • cron • Python • S3 Sync • Versionamento S3

Serviços EBS + S3 + EC2	Duração Aproximadamente 45 minutos	Instâncias EC2 Command Host + Processor
-----------------------------------	--	---

1. Visão Geral

Este laboratório aborda dois aspectos do gerenciamento de armazenamento na AWS: a automação de snapshots de volumes EBS (incluindo política de retenção com script Python) e a sincronização de arquivos entre um volume EBS e um bucket S3 — com versionamento para recuperar arquivos excluídos.

O ambiente tem duas instâncias: a Command Host, usada para administrar os recursos via AWS CLI, e a Processor, que possui o volume EBS a ser gerenciado.

1.1 Objetivos

- Criar um bucket S3 e associar um perfil IAM à instância Processor.
- Gerar um snapshot manual do volume EBS da Processor.
- Agendar criação automática de snapshots via cron (a cada minuto para o lab).
- Executar script Python que mantém apenas os 2 últimos snapshots.
- Sincronizar arquivos locais com bucket S3 usando aws s3 sync.
- Recuperar arquivo excluído usando versionamento do S3.

1.2 Fluxo Geral

#	Fase	O que foi feito
1	Configuração	Bucket S3 criado, perfil IAM S3BucketAccess associado à Processor
2	Snapshot manual	Volume EBS identificado, Processor parada, snapshot criado e verificado
3	Cron automático	Snapshots gerados a cada minuto via crontab
4	Retenção Python	snapshotter_v2.py executado: apenas os 2 últimos snapshots mantidos
5	S3 Sync	Arquivos sincronizados com aws s3 sync, depois re-sincronizados com --delete
6	Versionamento	file1.txt excluído, recuperado via list-object-versions + get-object

2. Tarefa 1 — Criar e Configurar Recursos

2.1 Criar o Bucket S3

1. No Console S3 > Criar bucket, definimos um nome único e mantivemos a Região padrão.
2. O nome do bucket será referenciado como s3-bucket-name ao longo do lab.

Nome do Bucket S3 — Ocultado

O nome real do bucket S3 é globalmente único e pode ser usado para identificar a conta. Foi omitido neste documento.

2.2 Associar Perfil IAM à Instância Processor

3. No Console EC2 > Instâncias > Processor > Ações > Segurança > Modificar função do IAM.
4. Selecionamos o perfil **S3BucketAccess** e clicamos em Atualizar função do IAM.

Por que associar o perfil IAM à Processor e não à Command Host?

A instância Processor é a que vai executar o aws s3 sync na Tarefa 3 (desafio). Ela precisa de permissões para acessar o bucket S3 e os volumes EBS. A Command Host já tinha acesso configurado para administrar os recursos via CLI.

3. Tarefa 2 — Gerar Snapshots da Instância

3.1 Conectar à Command Host

5. Conectamos à instância Command Host via EC2 Instance Connect.

3.2 Gerar o Snapshot Inicial

Todas as operações de snapshot foram realizadas a partir da Command Host. A sequência é: identificar o volume → parar a instância → criar snapshot → aguardar → reiniciar.

1

Identificar o VolumeId do EBS da Processor

Obtivemos o ID do volume através da tag Name=Processor:

```
aws ec2 describe-instances \
  --filter 'Name=tag:Name,Values=Processor' \
  --query 'Reservations[0].Instances[0].BlockDeviceMappings[0].Ebs.
{VolumeId:VolumeId}'
```

2

Obter o InstanceId da Processor

Necessário para parar a instância antes do snapshot:

```
aws ec2 describe-instances \
  --filters 'Name=tag:Name,Values=Processor' \
```

```
--query 'Reservations[0].Instances[0].InstanceId'
```

3

Parar a Processor e aguardar

Paramos a instância e usamos o waiter para aguardar o estado stopped:

```
aws ec2 stop-instances --instance-ids <INSTANCE-ID>
aws ec2 wait instance-stopped --instance-id <INSTANCE-ID>
```

4

Criar o snapshot e aguardar a conclusão

Criamos o snapshot e esperamos ele ser concluído antes de reiniciar:

```
aws ec2 create-snapshot --volume-id <VOLUME-ID>
aws ec2 wait snapshot-completed --snapshot-id <SNAPSHOT-ID>
```

5

Reiniciar a Processor

Com o snapshot concluído, reiniciamos a instância:

```
aws ec2 start-instances --instance-ids <INSTANCE-ID>
```

Por que parar a instância antes do snapshot?

Parar a instância garante que não há operações de escrita em andamento no volume no momento da captura — produzindo um snapshot consistente. É possível tirar snapshots com a instância rodando, mas há risco de capturar dados em estado intermediário.

3.3 Agendar Snapshots Automáticos com cron

Para gerar múltiplos snapshots rapidamente (para testar a política de retenção), configuramos um cron job que cria um snapshot a cada minuto:

```
echo "* * * * * aws ec2 create-snapshot --volume-id <VOLUME-ID> 2>&1 >>
/tmp/cronlog" > cronjob
crontab cronjob
```

Aguardamos alguns minutos e verificamos os snapshots gerados:

```
aws ec2 describe-snapshots --filters "Name=volume-id,Values=<VOLUME-ID>"
```

6. Após acumular vários snapshots, removemos o cron job:

```
crontab -r
```

Sintaxe do cron: * * * * *

Os 5 campos do cron representam: minuto (0-59), hora (0-23), dia do mês (1-31), mês (1-12) e dia da semana (0-7). O valor * significa 'qualquer'. Para produção, snapshots diários à meia-noite seriam: 0 0 * * *. Os snapshots do laboratório usam * * * * * (a cada minuto) apenas para acelerar a demonstração.

3.4 Aplicar Política de Retenção com snapshotter_v2.py

O script Python já estava disponível na instância. Ele localiza todos os volumes EBS da conta, gera um novo snapshot de cada um e mantém apenas os 2 últimos (excluindo os demais).

7. Visualizamos o script antes de executar:

```
more /home/ec2-user/snapshotter_v2.py
```

8. Verificamos os IDs dos snapshots existentes antes de executar:

```
aws ec2 describe-snapshots \
  --filters "Name=volume-id, Values=<VOLUME-ID>" \
  --query 'Snapshots[*].SnapshotId'
```

9. Executamos o script:

```
python3 snapshotter_v2.py
```

O script listou todos os snapshots excluídos. Após a execução, verificamos que apenas 2 IDs restaram:

```
aws ec2 describe-snapshots \
  --filters "Name=volume-id, Values=<VOLUME-ID>" \
  --query 'Snapshots[*].SnapshotId'
```

Política de Retenção Aplicada

De vários snapshots acumulados, o script manteve exatamente os 2 mais recentes, excluindo todos os demais. O número de IDs retornados passou de múltiplos para 2.

Volumeld, Instancelid e Snapshotids — Ocultados

Os IDs de volume, instância e snapshots são identificadores sensíveis da conta AWS e foram omitidos neste documento.

4. Tarefa 3 (Desafio) — Sincronizar Arquivos com o Amazon S3

O desafio consistiu em sincronizar arquivos de um diretório local na instância Processor com o bucket S3, ativar o versionamento e recuperar um arquivo excluído usando o historial de versões.

4.1 Preparar os Arquivos na Processor

10. Conectamos à instância Processor via EC2 Instance Connect.
11. Baixamos e descompactamos os arquivos de exemplo:

```
wget https://aws-tc-largeobjects.s3.us-west-2.amazonaws.com/[...]/files.zip
unzip files.zip
```

O zip continha a pasta `files/` com 3 arquivos: `file1.txt`, `file2.txt` e `file3.txt`.

4.2 Ativar Versionamento no Bucket S3

O versionamento precisa ser ativado antes de qualquer upload, para que as versões dos arquivos sejam preservadas:

```
aws s3api put-bucket-versioning \
  --bucket <s3-bucket-name> \
  --versioning-configuration Status=Enabled
```

4.3 Sincronizar com S3 Sync

1

Upload inicial dos 3 arquivos

Sincronizamos a pasta local files/ com o bucket:

```
aws s3 sync files s3://<s3-bucket-name>/files/
```

Saída: upload de file1.txt, file2.txt e file3.txt confirmado.

```
aws s3 ls s3://<s3-bucket-name>/files/ # verificar
```

2

Excluir file1.txt localmente

Simulamos a exclusão de um arquivo:

```
rm files/file1.txt
```

3

Sync com --delete para propagar a exclusão

A flag --delete faz o S3 excluir os arquivos que existem no bucket mas não no diretório local:

```
aws s3 sync files s3://<s3-bucket-name>/files/ --delete
```

Saída: delete: s3://<bucket>/files/file1.txt

```
aws s3 ls s3://<s3-bucket-name>/files/ # só 2 arquivos restam
```

4.4 Recuperar o Arquivo Excluído via Versionamento

1

Listar versões disponíveis

O versionamento preservou a versão anterior de file1.txt. Listamos as versões para obter o VersionId:

```
aws s3api list-object-versions \
  --bucket <s3-bucket-name> \
  --prefix files/file1.txt
```

A saída mostrou dois blocos: DeleteMarkers (indicação da exclusão) e Versions (a versão original do arquivo com seu VersionId).

2

Baixar a versão anterior

Usamos o VersionId da versão original para baixar o arquivo de volta:

```
aws s3api get-object \  
  --bucket <s3-bucket-name> \  
  --key files/file1.txt \  
  --version-id <VERSION-ID> \  
  files/file1.txt
```

```
ls files # confirmar que os 3 arquivos estão presentes
```

3

Re-sincronizar com o S3

Com file1.txt restaurado localmente, sincronizamos novamente:

```
aws s3 sync files s3://<s3-bucket-name>/files/  
aws s3 ls s3://<s3-bucket-name>/files/ # 3 arquivos novamente
```

✓ Arquivo Recuperado com Sucesso

O versionamento do S3 preservou a versão original de file1.txt mesmo após ele ter sido excluído com --delete. O workflow get-object (com VersionId) + sync restaurou o arquivo tanto localmente quanto no bucket.

💡 Por que não há um comando direto para restaurar versões no S3?

O S3 não tem um comando 'restore version' nativo. A abordagem correta é: (1) listar versões com list-object-versions, (2) baixar a versão desejada com get-object + --version-id, (3) re-enviar para o bucket com put-object ou sync. O delete marker pode ser removido diretamente com delete-object --version-id <id-do-marker> para uma abordagem alternativa.

5. Resumo dos Comandos Utilizados

Comando	Finalidade
aws ec2 describe-instances	Obter VolumeId e InstanceId da Processor por tag
aws ec2 stop-instances	Parar a instância antes do snapshot
aws ec2 wait instance-stopped	Aguardar a instância parar completamente
aws ec2 create-snapshot	Criar snapshot de um volume EBS
aws ec2 wait snapshot-completed	Aguardar o snapshot ser concluído
aws ec2 start-instances	Reiniciar a instância após o snapshot
aws ec2 describe-snapshots	Listar snapshots de um volume
crontab (cron job)	Agendar criação automática de snapshots
python3 snapshotter_v2.py	Aplicar política de retenção (manter 2 últimos)

<code>aws s3api put-bucket-versioning</code>	Ativar versionamento no bucket S3
<code>aws s3 sync</code>	Sincronizar diretório local com bucket S3
<code>aws s3 sync --delete</code>	Sync propagando exclusões para o S3
<code>aws s3api list-object-versions</code>	Listar versões de um objeto no S3
<code>aws s3api get-object --version-id</code>	Baixar versão específica de um objeto S3

6. Lições Aprendidas



Snapshots EBS devem ser tirados com a instância parada para garantir consistência. Em cenários de produção onde a parada não é viável, considere usar o Amazon Data Lifecycle Manager para gerenciar snapshots com consistência de aplicação.



O cron é simples mas poderoso para automatizar tarefas recorrentes no Linux. Em produção, o AWS Data Lifecycle Manager é a alternativa gerenciada e mais robusta para políticas de snapshot.



Scripts Python para gerenciar políticas de retenção são flexíveis. O mesmo padrão (listar, ordenar por data, excluir os mais antigos) pode ser adaptado para qualquer número de snapshots a manter.



`aws s3 sync` é idempotente: executar várias vezes só transfere o que mudou. A flag `--delete` deve ser usada com cuidado: ela propaga exclusões locais para o S3, o que é intencional em deploys mas pode ser destrutivo em backups.



O versionamento do S3 é a rede de segurança definitiva. Com ele ativo, nenhum arquivo é realmente perdido — até delete markers podem ser revertidos. Deve ser ativado em todos os buckets que armazenam dados importantes.



Versionamento do S3 tem custo: cada versão ocupa espaço e gera cobrança. Configure regras de ciclo de vida (S3 Lifecycle Rules) para expirar versões antigas automaticamente e controlar os custos.

7. Recursos Adicionais

- Comando AWS CLI: `aws s3api put-bucket-versioning` — docs.aws.amazon.com/cli/latest/reference/s3api/put-bucket-versioning.html
- Comando AWS CLI: `aws s3 sync` — docs.aws.amazon.com/cli/latest/reference/s3/sync.html
- Comando AWS CLI: `aws s3api list-object-versions` — docs.aws.amazon.com/cli/latest/reference/s3api/list-object-versions.html
- Comando AWS CLI: `aws s3api get-object` — docs.aws.amazon.com/cli/latest/reference/s3api/get-object.html
- AWS Training and Certification — aws.amazon.com/training

Documentação elaborada após a conclusão do laboratório. Dados sensíveis (nome do bucket S3, `Volumeld`, `InstanceId`, `SnapshotIds`, `VersionId`) foram propositalmente omitidos.